

SCS 1309

Database Management Systems

Lecture 9 – Relational Algebra and Calculus

MVPT Lakshika,
Lecturer (Probationary),
Department of Information Systems Engineering, UCSC.
Contact: tlv@ucsc.cmb.ac.lk

Outline

1. Relational Algebra

- Unary Operations: Selection, Projection, and Renaming
- Set Operations
- Joins and Division
- Aggregation and Grouping Operations

2. Relational Calculus

- Tuple relational calculus
- Domain relational calculus

Relational Query Languages

Two mathematical Query Languages form the basis for **real** query languages (e.g. SQL), and for implementation:

- **Relational Algebra:** More operational, very useful for representing execution plans. Basis for SEQUEL.
- **Relational Calculus:** Describe WHAT they want, rather than HOW to compute it (declarative). Basis for QUEL.

Relational Algebra

Introduction

- The relational algebra is a **theoretical language**.
 - Primarily serves as the basis for relational databases and SQL.
- Its main goal is to **define operators that convert one or more input relations into an output relation**, without changing the original relation(s).
 - Transform multiple input relations (the data stored in the database) into a single output relation (the query results).
- Relational algebra is a **procedural query language**.
 - It gives a step-by-step process to obtain the result of the query.
- It is also a **conceptual language**.
 - The queries made in it are **not run** on the computer.
 - However, this knowledge allows you to **understand the optimization and query execution** of RDBMS.

Introduction

- Relational algebra comprises a **set of operations**, such as **filtering** and **combining data**, to manipulate and query data in a relational database.
- The relational algebra is a relation-at-a-time (or set) language in which all tuples, possibly from several relations, are manipulated in one statement without looping.
- This is a purely mathematical framework.
 - Does not use English keywords; its operators are represented by symbols.
- The five **fundamental operations** in relational algebra: *Selection*, *Projection*, *Cartesian product*, *Union*, and *Set difference*.
- In addition, there are also the *Join*, *Intersection*, and *Division* operations, which can be expressed in terms of the five basic operations.

Unary Operations

Selection, Projection, Renaming

Unary Operations

- **Unary operations** in relational algebra are operations that act on a **single relation** (table) to produce a new relation as a result.
- The three fundamental unary operations are **selection** (σ), **projection** (π), and **renaming** (ρ).
- The other operations in relational algebra work on pairs of relations and are therefore called **binary operations**.
- In the following definitions, let **R** and **S** be two relations defined over the attributes $A = (a_1, a_2, \dots, a_N)$ and $B = (b_1, b_2, \dots, b_M)$, respectively.

Selection (σ)

- The **Selection** operation works on a single relation **R** and defines a relation that contains only those tuples of **R** that satisfy the specified condition (**predicate**).

Notation:

$\sigma_{\text{predicate}}(\mathbf{R})$

- It basically allows you to retrieve required tuples of the relations that match the condition as per the condition passed during the SQL Query.
- The selection operator only selects the required tuples but **does not display them**. For display, the data projection operator is used.
- A predicate is a **propositional logic** formula that may use connectors like AND, OR, and NOT and relational operators like $=$, $\neq$, $\geq$, $<$, $>$, $\leq$.



(a) Selection

Selection (σ)

Notation:

$\sigma_{\text{predicate}}(R)$

Example 1: List all staff with a salary greater than 10,000.

$\sigma_{\text{salary} > 10000}(\text{Staff})$

- Here, the **input relation (R)** is **Staff**, and the **predicate** is **salary > 10000**.

Output: A relation containing only those Staff tuples with a salary greater than 10,000.

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000	B003
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003

Selection (σ)

Logical Operators in Relational Algebra

- More complex **predicates** can be generated using the **logical operators** $\wedge$ (AND), $\vee$ (OR) and $\neg$ (NOT).
- Similar to how SQL uses **AND**, **OR**, and **NOT**.

Operator	Symbol	Meaning	Use in Relational Algebra	SQL Equivalent
AND	$\wedge$	Both conditions must be true	$\sigma_{\{\text{condition1} \wedge \text{condition2}\}}$ (Relation)	WHERE condition1 AND condition2
OR	$\vee$	At least one condition must be true	$\sigma_{\{\text{condition1} \vee \text{condition2}\}}$ (Relation)	WHERE condition1 OR condition2
NOT	$\neg$	Negates a condition	$\sigma_{\{\neg \text{condition}\}}$ (Relation)	WHERE NOT condition

Selection (σ)

Logical Operators in Relational Algebra

Example 2: List all details of properties located in the city of 'Glasgow' and the monthly rent is less than or equal to 450.

$$\sigma_{city='Glasgow' \wedge rent \leq 450}(PropertyForRent)$$

Example 3: List all properties that are either *Flats* with a rent less than 400 or any property with more than 5 rooms.

$$\sigma_{(type='Flat' \wedge rent < 400) \vee (rooms > 5)}(PropertyForRent)$$

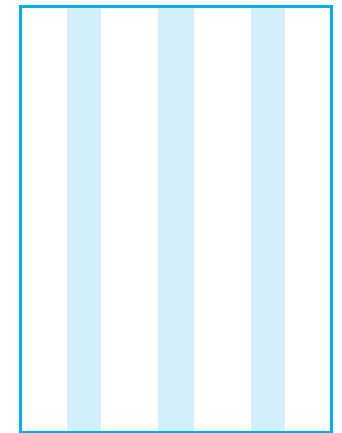
Projection(Π)

- The **Projection** operation works on a single relation **R** and defines a relation that contains a vertical subset of **R**, extracting the values of specified attributes and eliminating duplicates.

Notation: $\Pi_{a_1, \dots, a_n}(R)$

where a_1, a_2, a_n are attribute names of relation **R**.

- It basically allows you to pick specific columns from a given relational table based on the given condition and ignore all the other remaining columns.
- While the **selection** operation works on **rows**, the **projection** operation of relational algebra works on **columns**.
- By default, projection **removes duplicate** data.



(b) Projection

Projection(Π)

Example 4: Produce a list of salaries for all staff, showing only the staffNo, fName, lName, and salary details.

$\Pi_{\text{staffNo, fName, lName, salary}}(\text{Staff})$

Notation:

$\Pi_{a_1, \dots, a_n}(R)$

Output: A relation that contains only the designated Staff attributes *staffNo*, *fName*, *lName*, and *salary*, in the specified order.

staffNo	fName	lName	salary
SL21	John	White	30000
SG37	Ann	Beech	12000
SG14	David	Ford	18000
SA9	Mary	Howe	9000
SG5	Susan	Brand	24000
SL41	Julie	Lee	9000

Activity 1

1. Write the Relational Algebra expression to retrieve staff members who are not Managers and are not assigned to branch 'B005'.

$$\sigma_{\neg (branchNo = 'B005') \wedge \neg (position = 'Manager')} (Staff)$$

2. Write the Relational Algebra expression to retrieve only the client numbers, first names, and last names of all clients.

$$\pi_{clientNo, fName, lName} (Client)$$

3. Write the Relational Algebra expression to retrieve the Property Number and City for all properties that have a rent higher than 500.

$$\pi_{propertyNo, city} (\sigma_{rent > 500} (PropertyForRent))$$

Rename operation

- Alternatively, you can use the **Rename operation** ρ (rho), which gives a name to the result of a relational algebra operation.

$$\rho(E) \text{ or } \rho_{S(a_1, a_2, \dots, a_n)}(E)$$

- The Rename operation provides a new name **S** for the expression **E**, and optionally names the attributes as $a_1, a_2, \dots, a_n$.
- Rename allows an optional name for each of the attributes of the new relation to be specified.

Example 5: Retrieve a list containing only the staffNo and salary of all employees, but wants the resulting table to be called "Payroll_List".

$$\rho_{\text{Payroll_List}} \left(\pi_{\text{staffNo, salary}}(\text{Staff}) \right)$$

Example 6: Rename the city column to "Branch_City" and resulting table as "Result".

$$\rho_{\text{Result}}(\text{Branch_City}) \left(\pi_{\text{city}}(\text{Branch}) \right)$$

Relational Algebra

Set Operations

Set operators

- The **Selection** and **Projection** operations extract information from **only one relation**.
- There are obviously cases where you need **to combine information from several relations**.
- **Set operations:**

Union	$R \cup S$	The union of two relations R and S defines a relation that contains all the tuples of R, or S, or both R and S, duplicate tuples being eliminated. R and S must be union-compatible.
Set difference	$R - S$	Defines a relation consisting of the tuples that are in relation R , but not in S . R and S must be union-compatible
Intersection	$R \cap S$	Defines a relation consisting of the set of all tuples that are in both R and S . R and S must be union-compatible.
Cartesian product	$R \times S$	Defines a relation that is the concatenation of every tuple of relation R with every tuple of relation S .

Set operators

- For **Union**, **Set difference**, and **Intersection**, the relations must be **union-compatible**.
 - These set operations are possible only if the schemas of the two relations match, that is, if they have the **same number of attributes**, with **each pair of corresponding attributes having the same domain**.
 - Note that attribute names are not used in defining union compatibility.
 - In some cases, the Projection operation may be used to make two relations union-compatible.

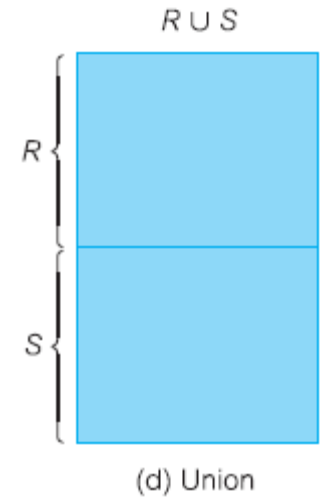
Union Operation

- If **R** and **S** have *I* and *J* tuples, respectively, their union is obtained by concatenating them into one relation with a maximum of (*I* + *J*) tuples.

Example 7: List all cities where there is either a branch office or a property for rent.

$$\Pi_{\text{city}}(\text{Branch}) \cup \Pi_{\text{city}}(\text{PropertyForRent})$$

- To produce union-compatible relations, first use the **Projection** operation to project the *Branch* and *PropertyForRent* relations over the attribute *city*, eliminating duplicates where necessary.
- Next, use the **Union** operation to combine these new relations to produce the final result.



Result:

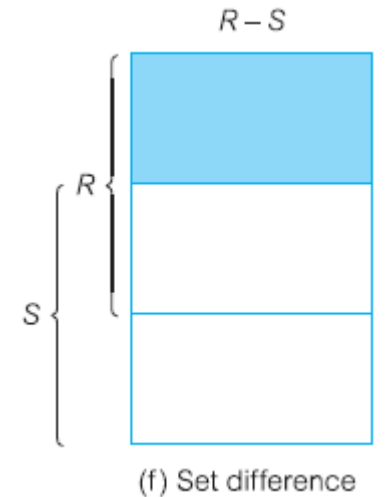
city
London
Aberdeen
Glasgow
Bristol

Set Difference

Example 8: List all cities where there is a branch office but no properties for rent.

$$\Pi_{\text{city}}(\text{Branch}) - \Pi_{\text{city}}(\text{PropertyForRent})$$

- First, produce union-compatible relations by **projecting** the *Branch* and *PropertyForRent* relations over the attribute *city*.
- Next, use the **Set difference** operation to combine these new relations to produce the result.



Result:

city
Bristol

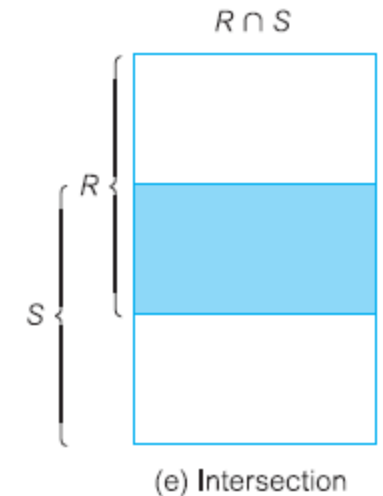
Intersection

Example 9: List all cities where there is both a branch office and at least one property for rent.

$$\Pi_{\text{city}}(\text{Branch}) \cap \Pi_{\text{city}}(\text{PropertyForRent})$$

- First, produce union-compatible relations by **projecting** the *Branch* and *PropertyForRent* relations over the attribute *city*.
- Next, use the **Intersection** operation to combine these new relations to produce the result.
- You can express the Intersection operation in terms of the **Set difference** operation:

$$\mathbf{R \cap S = R - (R - S)}$$

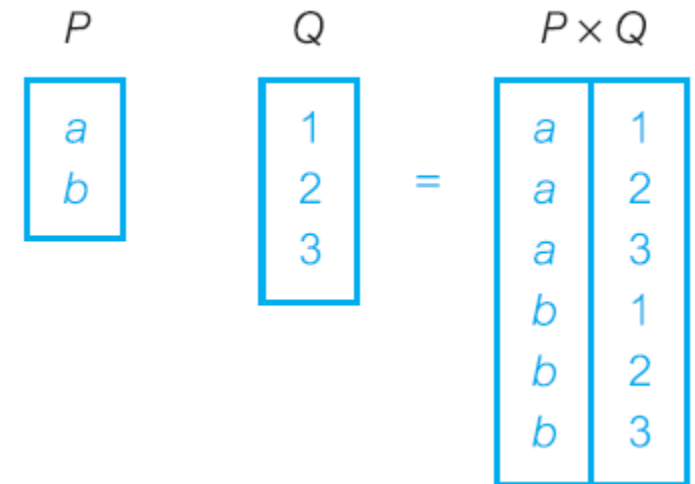


Result:

city
Aberdeen
London
Glasgow

Cartesian product

- The **Cartesian product** operation **multiplies two relations** to define another relation consisting of all possible pairs of tuples from the two relations.
- Therefore, if one relation has ***I*** tuples and ***N*** attributes and the other has ***J*** tuples and ***M*** attributes, the Cartesian product relation will contain **$(I * J)$** tuples with **$(N + M)$** attributes.
- It is possible that the two relations may have attributes with the same name.
- In this case, the attribute names are prefixed with the relation name to maintain the uniqueness of attribute names within a relation.



Cartesian product

Example 10: List the names and comments of all clients who have viewed a property for rent.

- The names of clients are held in the *Client* relation, and the details of viewings are held in the *Viewing* relation.
- To obtain the list of clients and the comments on properties they have viewed, you need to combine these two relations:

$$(\Pi_{\text{clientNo, fName, lName}}(\text{Client})) \times (\Pi_{\text{clientNo, propertyNo, comment}}(\text{Viewing}))$$

- The first tuple of this relation contains different **clientNo** values.
- To obtain the required list, you need to carry out a **Selection** operation on this relation to extract those tuples where **Client.clientNo = Viewing.clientNo**.

$$\sigma_{\text{Client.clientNo} = \text{Viewing.clientNo}}((\Pi_{\text{clientNo, fName, lName}}(\text{Client})) \times (\Pi_{\text{clientNo, propertyNo, comment}}(\text{Viewing})))$$

Decomposing complex operations

- The relational algebra operations can be of arbitrary complexity.
- You can **decompose** such operations into a series of smaller relational algebra operations and give a name to the results of intermediate expressions.
- Use the **assignment operation**, denoted by $\leftarrow$, to name the results of a relational algebra operation.

Example 11: In the previous example, you could rewrite the operation as follows:

TempViewing (clientNo, propertyNo, comment) $\leftarrow \Pi_{\text{clientNo, propertyNo, comment}}(\text{Viewing})$

TempClient (clientNo, fName, lName) $\leftarrow \Pi_{\text{clientNo, fName, lName}}(\text{Client})$

Comment (clientNo, fName, lName, vclientNo, propertyNo, comment) $\leftarrow \text{TempClient} \times \text{TempViewing}$

Result $\leftarrow \sigma_{\text{clientNo} = \text{vclientNo}}(\text{Comment})$

Activity 2

1. Create a single mailing list (including fName, lName, and telNo) of every person in the database (both Clients and Owners).

$$\pi_{fName, lName, telNo}(Client) \cup \pi_{fName, lName, telNo}(PrivateOwner)$$

2. Find the first and last names of people who appear in both the *Client* table and the *PrivateOwner* table.

$$\pi_{fName, lName}(Client) \cap \pi_{fName, lName}(PrivateOwner)$$

3. Rewrite the answer for Question 2 without using the $\cap$. Use only the Set Difference (-) symbol.

$$R \leftarrow \pi_{fName, lName}(Client) \qquad S \leftarrow \pi_{fName, lName}(PrivateOwner)$$

$$R - (R - S)$$

Relational Algebra

Joins And Divisions

Join Operations

- Typically, we want only combinations of the Cartesian product that satisfy certain conditions, and so we would normally use a **Join operation** instead of the Cartesian product operation.
- The Join operation combines two relations to form a new relation.
- Join is a derivative of the Cartesian product, equivalent to performing a Selection operation, using the join predicate as the selection formula, over the Cartesian product of the two operand relations.
- Join is one of the most difficult operations to implement efficiently in an RDBMS and is one of the reasons why relational systems have intrinsic performance problems.

Join Operations

- There are **various forms of Join operation**, each with subtle differences, some more useful than others:

- Theta join / Conditional Join
- Equijoin (a particular type of Theta join)
- Natural join
- Semi join
- Outer join

} **Inner Join types**

T	
A	B
a	1
b	2

U	
B	C
1	x
1	y
3	z

$T \bowtie U$		
A	B	C
a	1	x
a	1	y

(g) Natural join

$T \bowtie_B U$	
A	B
a	1

(h) Semijoin

$T \bowtie_C U$		
A	B	C
a	1	x
a	1	y
b	2	

(i) Left Outer join

Inner Join: Theta join

- The **Theta join (Θ -join)** operation defines a relation that contains tuples satisfying the **predicate (F)** from the Cartesian product of **R** and **S**.

$$\mathbf{R} \bowtie_F \mathbf{S}$$

- The **predicate F** is of the form $R.a_i \ \Theta \ S.b_i$ where Θ may be one of the comparison operators ($<, \leq, >, \geq, =, \neq$).
- You can rewrite the Theta join in terms of the basic **Selection** and **Cartesian** product operations:

$$\mathbf{R} \bowtie_F \mathbf{S} = \sigma_F (\mathbf{R} \times \mathbf{S})$$

- As with the Cartesian product, the degree (**number of attributes**) of a Theta join is the sum of the degrees of the operand relations **R** and **S**.

Inner Join: Equijoin

- In the case where the predicate F contains only equality ($=$), the term **Equijoin** is used instead.

Example 12: List the names and comments of all clients who have viewed a property for rent.

- Using the Cartesian product and Selection operations:

$$\sigma_{\text{Client.clientNo} = \text{Viewing.clientNo}}((\pi_{\text{clientNo, fName, lName}}(\text{Client})) \times (\pi_{\text{clientNo, propertyNo, comment}}(\text{Viewing})))$$

- The same result is obtained using the Equijoin operation:

$$(\pi_{\text{clientNo, fName, lName}}(\text{Client})) \bowtie_{\text{Client.clientNo} = \text{Viewing.clientNo}} (\pi_{\text{clientNo, propertyNo, comment}}(\text{Viewing}))$$

or

Result $\leftarrow$ **TempClient** $\bowtie_{\text{TempClient.clientNo} = \text{TempViewing.clientNo}}$ **TempViewing** (Using assignment operation)

Inner Join: Natural join

- The **Natural join** is an Equijoin of the two relations **R** and **S** over all common attributes **x**.

$$R \bowtie S$$

- One occurrence of each common attribute is eliminated from the result.
- The **Natural join** operation performs an Equijoin over all the attributes in the two relations that have the same name.
- The degree of a Natural join is the sum of the degrees of the relations R and S less the number of attributes in x.

Inner Join: Natural join

Example 13: List the names and comments of all clients who have viewed a property for rent.

- Earlier, you used the **Equijoin** to produce this list, but the resulting relation had two occurrences of the join attribute *clientNo*.

$(\Pi_{\text{clientNo, fName, lName}}(\text{Client})) \bowtie_{\text{Client.clientNo} = \text{Viewing.clientNo}} (\Pi_{\text{clientNo, propertyNo, comment}}(\text{Viewing}))$

- We can use the **Natural join** to remove one occurrence of the *clientNo* attribute:

$(\Pi_{\text{clientNo, fName, lName}}(\text{Client})) \bowtie (\Pi_{\text{clientNo, propertyNo, comment}}(\text{Viewing}))$

or

Result $\leftarrow$ TempClient $\bowtie$ TempViewing (Using assignment operation)

Outer join

- Often in joining two relations, a tuple in one relation does not have a matching tuple in the other relation (there is no matching value in the join attributes)
- We may want tuples from one of the relations to appear in the result even when there are no matching values in the other relation.
- This may be accomplished using the [Outer join](#).
- Missing values in the second relation are set to *null*.
- **Left (natural) Outer join:** $R \bowtie S$, as it keeps every tuple in the left-hand relation in the result.
- **Right Outer join:** $R \bowtie S$, that keeps every tuple in the right-hand relation in the result.
- **Full Outer join:** $R \bowtie S$, that keeps all tuples in both relations, padding tuples with nulls when no matching tuples are found.

Outer join

Example 14: Produce a status report on property viewings.

- In this case, we want to produce a relation consisting of the properties that have been viewed with comments and those that have not been viewed.
- This can be achieved using the following Outer join:

$(\Pi_{\text{propertyNo, street, city}}(\text{PropertyForRent})) \bowtie \text{Viewing}$

- Note that properties PL94, PG21, and PG16 have no viewings, but these tuples are still contained in the result with nulls for the attributes from the Viewing relation.

propertyNo	street	city	clientNo	viewDate	comment
PA14	16 Holhead	Aberdeen	CR56	24-May-04	too small
PA14	16 Holhead	Aberdeen	CR62	14-May-04	no dining room
PL94	6 Argyll St	London	null	null	null
PG4	6 Lawrence St	Glasgow	CR76	20-Apr-04	too remote
PG4	6 Lawrence St	Glasgow	CR56	26-May-04	
PG36	2 Manor Rd	Glasgow	CR56	28-Apr-04	
PG21	18 Dale Rd	Glasgow	null	null	null
PG16	5 Novar Dr	Glasgow	null	null	null

Semi join

- The **Semijoin** operation defines a relation that contains the tuples of **R** that participate in the join of R with **S**.

$$R \triangleright_F S$$

- The Semijoin operation performs a join of the two relations and then projects over the attributes of the first operand.
- Advantage: Decreases the number of tuples that need to be handled to form the join.
- It is particularly useful for computing joins in distributed systems.
- We can rewrite the Semijoin using the Projection and Join operations:

$$R \triangleright_F S = \Pi_A(R \bowtie S) \quad \text{A is the set of all attributes for R}$$

- This is actually a Semi-Theta join. There are variants for Semi-Equijoin and Semi-Natural join.

Semi join

Example 15: List complete details of all staff who work at the branch in Glasgow.

- If you are interested in seeing only the attributes of the *Staff* relation, you can use the following **Semijoin** operation.

Staff $\triangleright$ Staff.branchNo = Branch.branchNo ($\sigma_{\text{city} = \text{'Glasgow'}}(\text{Branch})$)

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000	B003
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003

Division Operation

- The **Division operation** is useful for a particular type of query that occurs quite frequently in database applications.
- Assume relation **R** is defined over the attribute set **A** and relation **S** is defined over the attribute set **B** such that $\mathbf{B} \subseteq \mathbf{A}$ (B is a subset of A).
- Let $\mathbf{C} = \mathbf{A} - \mathbf{B}$, that is, **C** is the set of attributes of **R** that are not attributes of **S**.
- The **Division operation** defines a relation over the attributes **C** that consists of the set of tuples from **R** that match the combination of **every** tuple in **S**.

$$\mathbf{R} \div \mathbf{S}$$

Division Operation

Example 16: Identify all clients who have viewed all properties with three rooms.

- We can use the **Selection operation** to find all properties with three rooms, followed by the **Projection** operation to produce a relation containing only these property numbers.
- We can then use the following Division operation to obtain the new relation.

$$(\Pi_{\text{clientNo, propertyNo}}(\text{Viewing})) \div (\Pi_{\text{propertyNo}}(\sigma_{\text{rooms} = 3}(\text{PropertyForRent})))$$

$\Pi_{\text{clientNo, propertyNo}}(\text{Viewing})$

clientNo	propertyNo
CR56	PA14
CR76	PG4
CR56	PG4
CR62	PA14
CR56	PG36

$\Pi_{\text{propertyNo}}(\sigma_{\text{rooms} = 3}(\text{PropertyForRent}))$

propertyNo
PG4
PG36

RESULT

clientNo
CR56

Relational Algebra

Aggregation and Grouping Operations

Aggregation Operation

- Applies the aggregate function list, **AL**, to the relation **R** to define a relation over the aggregate list. **AL** contains one or more (**<aggregate_function> <attribute>**) pairs.

$$\mathcal{S}_{AL}(R)$$

- The main aggregate functions are:
 - COUNT – returns the number of values in the associated attribute.
 - SUM – returns the sum of the values in the associated attribute.
 - AVG – returns the average of the values in the associated attribute.
 - MIN – returns the smallest value in the associated attribute.
 - MAX – returns the largest value in the associated attribute.

Aggregation Operation

Example 14: How many properties cost more than 350 per month to rent?

$\rho_R(\text{myCount}) \bowtie_{\text{COUNT propertyNo}} (\sigma_{\text{rent} > 350} (\text{PropertyForRent}))$

This expression produces the relation **R**.

myCount
5

Example 15: Find the minimum, maximum, and average staff salary.

- We can use the aggregate functions, MIN, MAX, and AVERAGE, to produce the relation

$\rho_R(\text{myMin}, \text{myMax}, \text{myAverage}) \bowtie_{\text{MIN salary, MAX salary, AVERAGE salary}} (\text{Staff})$

This expression produces the relation **R**.

myMin	myMax	myAverage
9000	30000	17000

Grouping Operation

- Groups the tuples of relation **R** by the grouping attributes, **GA**, and then applies the aggregate function list **AL** to define a new relation.
- **AL** contains one or more (**<aggregate_function> <attribute>**) pairs.
- The resulting relation contains the grouping attributes, **GA**, along with the results of each of the aggregate functions.

$$GA \mathfrak{S}_{AL}(R)$$

- The general form of the grouping operation is as follows:

$$a_1, a_2, \dots, a_n \mathfrak{S}_{\langle A_p a_p \rangle, \langle A_q a_q \rangle, \dots, \langle A_z a_z \rangle}(R)$$

where **R** is any relation, **a₁, a₂, . . . , a_n** are attributes of **R** on which to group, **a_p, a_q, . . . , a_z** are other attributes of **R**, and **A_p, A_q, . . . , A_z** are aggregate functions.

- The tuples of **R** are partitioned into groups such that:
 - all tuples in a group have the same value for **a₁, a₂, . . . , a_n**;
 - tuples in different groups have different values for **a₁, a₂, . . . , a_n**.

Grouping Operation

Example 16: Find the number of staff working in each branch and the sum of their salaries.

- First, need to group tuples according to the branch number (*branchNo*).
- Next, use the aggregate functions COUNT and SUM to produce the required relation.

$\rho_R(\text{branchNo, myCount, mySum})_{\text{branchNo}} \bowtie \text{COUNT staffNo, SUM salary}(\text{Staff})$

branchNo	myCount	mySum
B003	3	54000
B005	2	39000
B007	1	9000

Activity 3

1. The Finance Department wants to know the total annual expenditure on staff salaries.

$$\mathfrak{I}_{SUM\ salary}(Staff)$$

2. The manager wants to see the number of properties available in each city.

$$city\ \mathfrak{I}_{COUNT\ propertyNo}(PropertyForRent)$$

3. Find the average rent of only the flats.

$$\mathfrak{I}_{AVG\ rent}\left(\sigma_{type='Flat'}(PropertyForRent)\right)$$

4. Find the total number of female staff members and their total combined salary.

$$\mathfrak{I}_{COUNT\ (staffNo),\ SUM\ (salary)}(\sigma_{sex='F'}(Staff))$$

Activity 3

5. Report the total number of staff and the highest salary for each Position (Manager, Assistant, etc.) within the B005 branch.

$$\text{position } \mathcal{F}_{COUNT(staffNo), MAX(salary)} \left(\pi_{position, salary, staffNo} \left(\sigma_{branchNo='B005'}(Staff) \right) \right)$$

6. The Marketing Department wants to analyze the Aberdeen market. They need a list of different property types (House, Flat, etc.) and the average rent for each, but they only want to include properties that have more than 3 rooms.

$$\begin{aligned} \mathbf{Result1} &\leftarrow \pi_{type, rent, propertyNo} \left(\sigma_{city='Aberdeen' \wedge rooms > 3}(PropertyForRent) \right) \\ &\text{type } \mathcal{F}_{AVG(rent), COUNT(propertyNo)}(\mathbf{Result1}) \end{aligned}$$

More...

Checking for NULL Values in Relational Algebra

- Relational Algebra does not have direct support for **NULL** values as it follows the relational model, where every tuple has a definite value.
- However, databases that support NULL values (like SQL) handle this differently.
- To simulate NULL checks in Relational Algebra, we typically use Outer Joins ($\bowtie$, $\bowtie$) and Set Difference ($-$).

ASC & DESC Ordering in Relational Algebra

- In basic relational algebra, there is **no built-in ordering operation**.
- Because relational algebra is based on set theory, and in mathematics, a set has no order.
- In extended relational algebra, a sorting operator (τ) is added.

$\tau_{\text{attribute list}}(\text{Relation})$

Summary - Relational Algebra Operations

Operation	Notation	Function
Selection	$\sigma_{\text{predicate}}(R)$	Produces a relation that contains only those tuples of R that satisfy the specified <i>predicate</i> .
Projection	$\Pi_{a_1, \dots, a_n}(R)$	Produces a relation that contains a vertical subset of R, extracting the values of specified attributes and eliminating duplicates.
Union	$R \cup S$	Produces a relation that contains all the tuples of R, or S, or both R and S, duplicate tuples being eliminated. R and S must be union-compatible.
Set difference	$R - S$	Produces a relation that contains all the tuples in R that are not in S. R and S must be union-compatible.
Intersection	$R \cap S$	Produces a relation that contains all the tuples in both R and S. R and S must be union-compatible.
Cartesian product	$R \times S$	Produces a relation that is the concatenation of every tuple of relation R with every tuple of relation S.

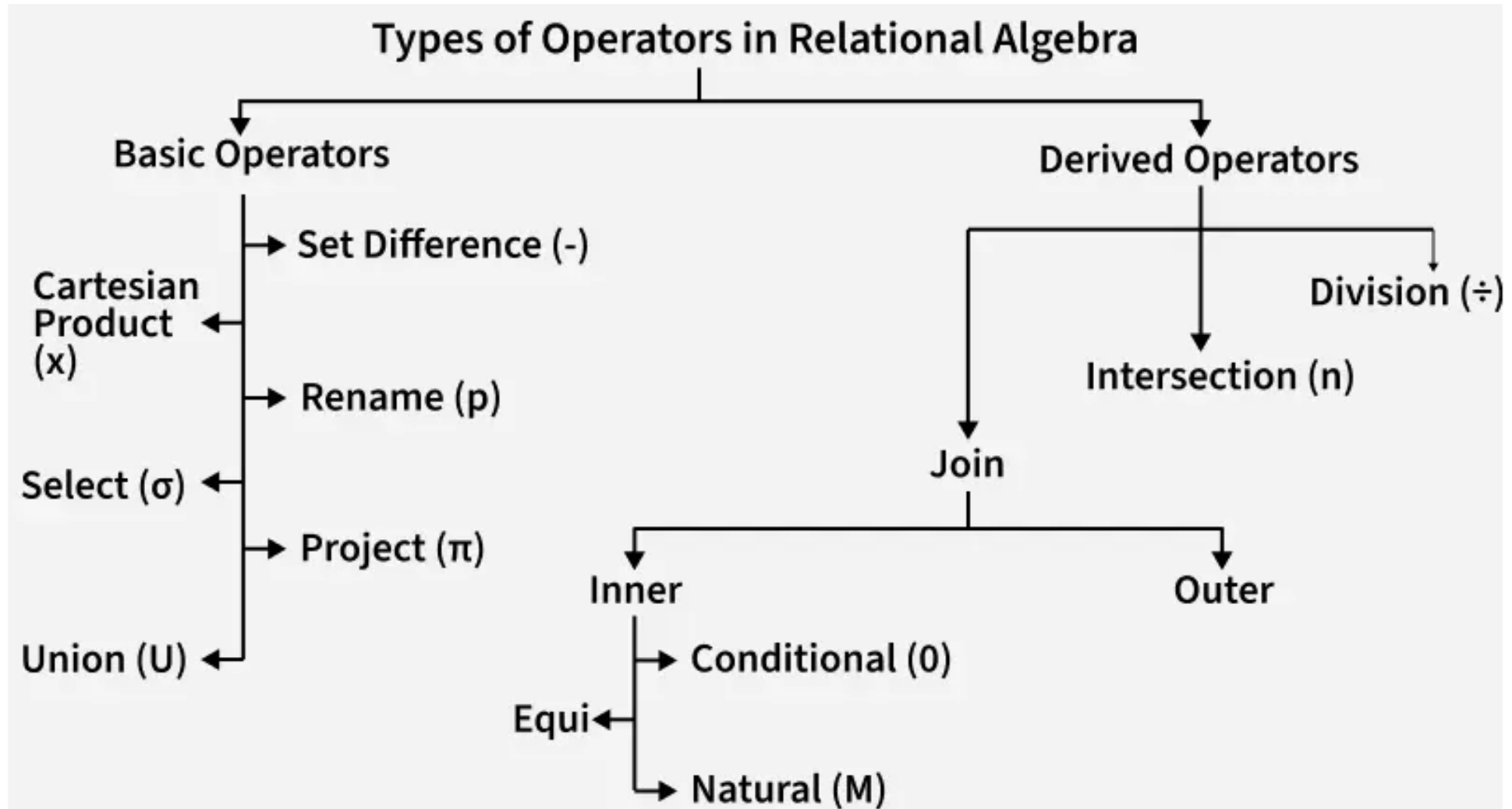
Summary of the Relational Algebra Operations

Operation	Notation	Function
Theta join	$R \bowtie_F S$	Produces a relation that contains tuples satisfying the predicate F from the Cartesian product of R and S .
Equijoin	$R \bowtie_F S$	Produces a relation that contains tuples satisfying the predicate F (which only contains equality comparisons) from the Cartesian product of R and S .
Natural join	$R \bowtie S$	An Equijoin of the two relations R and S over all common attributes x . One occurrence of each common attribute is eliminated.
(Left) Outer join	$R \rhd S$	A join in which tuples from R that do not have matching values in the common attributes of S are also included in the result relation.
Semijoin	$R \bowtie_F S$	Produces a relation that contains the tuples of R that participate in the join of R with S satisfying the predicate F .
Division	$R \div S$	Produces a relation that consists of the set of tuples from R defined over the attributes C that match the combination of every tuple in S , where C is the set of attributes that are in R but not in S .

Summary of the Relational Algebra Operations

Operation	Notation	Function
Aggregate	$\mathfrak{S}_{AL}(R)$	Applies the aggregate function list, AL, to the relation R to define a relation over the aggregate list. AL contains one or more (<aggregate_function>, <attribute>) pairs.
Grouping	$_{GA}\mathfrak{S}_{AL}(R)$	Groups the tuples of relation R by the grouping attributes, GA, and then applies the aggregate function list AL to define a new relation. AL contains one or more (<aggregate_function>, <attribute>) pairs. The resulting relation contains the grouping attributes, GA, along with the results of each of the aggregate functions.

Summary - Relational Algebra Operations



Any Questions?